

Availability & Reliability

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

Phase 1 briefly defined SLI/SLO/SLA and mentioned "nines" of uptime. This chapter unpacks what actually makes a system available and reliable in the first place, because these two words, though often used interchangeably, mean genuinely different things and interviewers expect you to keep them separate.

Reliability is about correctness under expected conditions: does the system do what it's supposed to do, consistently, without producing wrong results or silent failures? A payment system that occasionally double-charges a customer is unreliable, even if it's technically "up" 99.99% of the time. **Availability** is narrower and more mechanical: is the system reachable and responding at all, measured as a percentage of time over some period (a month, a year)? A system can be perfectly reliable when it's up, but still have poor availability if it goes down often.

The core enemy of both is the **single point of failure (SPOF)** — any one component whose failure takes down the whole system. A single application server, a single database instance, a single network link, even a single cloud region, are all SPOFs if nothing backs them up. The entire discipline of designing for availability is really just the discipline of systematically hunting down and eliminating SPOFs, one layer at a time: multiple app servers behind a load balancer removes the app-server SPOF (Phase 1 Chapter 9); a read replica plus a promotion mechanism removes the single-database SPOF; multiple availability zones removes the single-data-center SPOF; multiple regions removes the single-region SPOF.

Backend engineer analogy

Think of your local dev environment where a single PHP-FPM worker or a single Node.js process handles requests — if it crashes on an unhandled exception, everything stops until you manually restart it. In production, a process manager (or Kubernetes) automatically respawns crashed workers and a load balancer routes around unhealthy ones. That automatic detect-and-recover loop, scaled up to entire servers and data centers, is exactly what availability engineering is.

Redundancy and failover

Redundancy is deliberately duplicating a component so a spare exists when the primary fails. **Failover** is the mechanism that actually detects a failure and switches traffic to the redundant component. These are two separate concerns: you can have redundancy without automated failover (a standby database someone manually promotes during an incident), and the quality of your failover mechanism — how fast and how automatic — directly determines how much downtime an actual failure causes.

Failover comes in two flavors worth naming precisely. **Hot failover** (or active-active) keeps the backup fully running and serving traffic already, so switching over is close to instantaneous — the cost is running duplicate capacity all the time. **Cold failover** (or active-passive) keeps the backup powered

down or idle until needed, which is cheaper to run but slower to activate, since the standby often needs to boot up, warm caches, and catch up on replicated data before it can safely serve production traffic.

It's worth being explicit about the actual math behind "nines." 99.9% ("three nines") availability allows roughly 8.7 hours of downtime per year. 99.99% ("four nines") allows roughly 52 minutes per year. 99.999% ("five nines") allows roughly 5 minutes per year. Each additional nine is not a linear increase in engineering effort — going from three nines to four nines, and especially four to five, typically requires disproportionately more redundancy, automation, and organizational discipline (on-call processes, chaos testing, blameless postmortems), which is exactly why interviewers expect you to justify *why* a given system needs five nines rather than assuming more nines is automatically better.

Reliability, separately, is improved through different tools: input validation, idempotent operations (Chapter 24 goes deep on this), retries with backoff, circuit breakers that stop hammering a failing downstream dependency, and thorough testing. A highly available system that's unreliable — always up, but frequently returning wrong answers — can be more dangerous than a less available but strictly correct one, especially for financial systems, which is why payment platforms tend to fail closed (reject the request) rather than fail open (guess and possibly corrupt state) when uncertain.

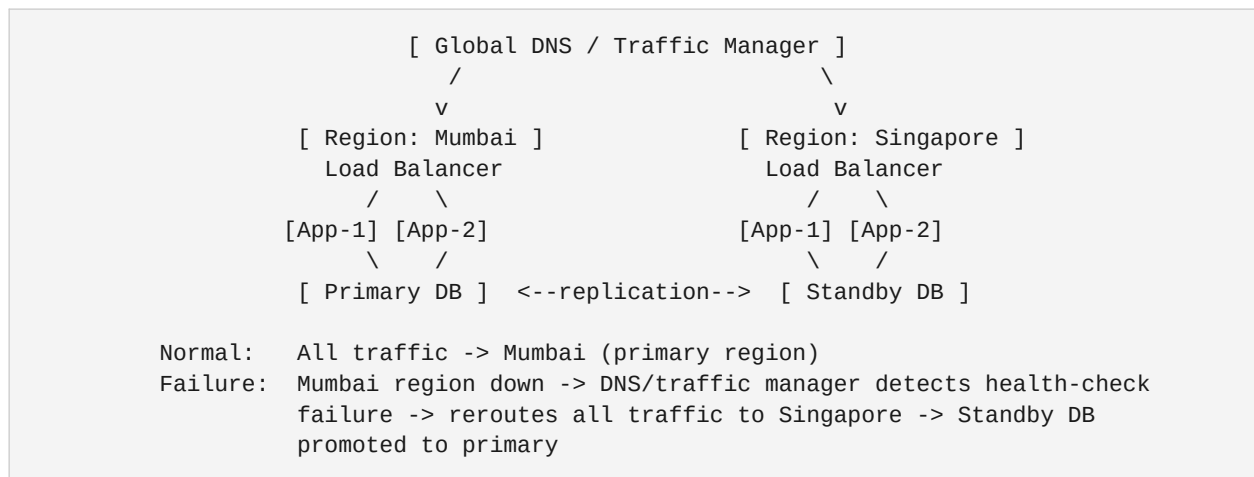
2. Real-World Example / Case Study

When Careem or Talabat runs across multiple UAE and Saudi data center regions, a regional outage — a power failure, a network partition, a cloud provider incident — doesn't have to mean total downtime. Traffic can be automatically rerouted to a healthy region via DNS-level or load-balancer-level failover, with users in the affected region experiencing perhaps a brief spike in latency (their requests now travel further) rather than a full outage.

Razorpay and PhonePe, handling live payment traffic, cannot tolerate a single database failure taking down transaction processing. Their database layer typically runs with a hot standby that can be promoted to primary within seconds of a detected failure, combined with careful reliability engineering around the payment logic itself so that even during a failover window, in-flight transactions aren't silently lost or double-processed — a direct preview of the idempotency guarantees Chapter 24 covers.

STC Pay and Tamara, operating in Saudi Arabia's fast-growing fintech and BNPL space, are held to strict regulatory expectations around both availability (uptime commitments to merchants and consumers) and reliability (accurate transaction records for compliance and audit). This is a good illustration of why the two concepts genuinely need separate design attention: a system can pass every uptime SLA while still failing a compliance audit over a handful of reliability bugs.

3. Diagram



4. Trade-offs Table

Approach	Availability gain	Cost / Complexity
Active-active (hot) failover	Near-instant failover, minimal downtime	Double infrastructure cost, cross-region data sync complexity
Active-passive (cold) failover	Lower cost than active-active	Slower recovery (minutes), risk of misconfigured standby
Multi-AZ within one region	Survives data-center-level failures cheaply	Doesn't survive a full regional outage
Multi-region deployment	Survives entire region failures	Significant added complexity: data sync, latency, cost
No redundancy (single instance)	Cheapest, simplest to operate	Any single failure is a full outage

5. Common Interview Questions

Q1: "What's the difference between availability and reliability? Give an example where a system is one but not the other."

Model approach: Define availability as "reachable and responding" and reliability as "correct when it responds." Example: a search engine that's always up but occasionally returns irrelevant or wrong results is highly available but unreliable; a batch payroll job that runs correctly once a month but sometimes fails to start on schedule is reliable (when it runs, it's correct) but has poor availability.

Q2: "Design a system to survive an entire data center going down."

Model approach: Start from a single-region, multi-AZ baseline, explain that surviving a full data-center loss requires redundant app servers and a replicated database spread across at least two independent availability zones, plus a load balancer or DNS layer capable of detecting the failure and rerouting traffic automatically, and be explicit about whether you're proposing hot or cold failover and why.

Q3: "How many nines of availability does this system actually need, and how did you decide?"

Model approach: Tie the number directly to business impact — a payments API serving live checkout traffic might justify four or five nines given the revenue and trust cost of downtime, while an internal

analytics dashboard might be fine at three nines. Reject the premise that more nines is free; name the added redundancy and cost each additional nine requires.

Q4: "What's the difference between hot and cold failover, and when would you choose each?"

Model approach: Hot failover keeps the backup live and serving traffic already, giving near-instant recovery at double the running cost; cold failover keeps the backup idle until needed, costing less but recovering slower. Choose hot for latency- and revenue-critical paths (checkout, payments); cold is often acceptable for less time-sensitive internal systems.

6. Practice Exercises

- 1 A ride-hailing app's matching service goes down for 40 minutes once a quarter. Calculate its approximate annual availability percentage and classify it as roughly how many 'nines.'
- 2 Propose a redundancy plan for a payment gateway's database that must survive both a single-server failure and a full data-center outage, and justify hot vs. cold failover for each layer.
- 3 Describe a real scenario where a system could be 100% available yet completely unreliable, and explain what engineering practices (not redundancy) would fix it.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Redundancy Explained Removing Single Points of Failure in System Design"	Direct treatment of the SPOF concept this chapter centers on.
English	"High Availability Eliminate Single Points of Failure System Design Concepts for Beginners"	Beginner-friendly walkthrough of HA design principles.
Hindi	"Scalability vs Availability vs Reliability Explained" — Gate Smashers	Hindi breakdown that cleanly separates these three commonly confused terms.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Places availability/reliability in context of the full Hindi HLD roadmap.

8. Key Takeaways

- Availability measures uptime ("is it reachable"); reliability measures correctness ("is it right when it responds").
- A system can be highly available but unreliable, or highly reliable but poorly available — they need separate design attention.
- Redundancy (spare components) and failover (the switch-over mechanism) are two distinct pieces of the same puzzle.
- Hot (active-active) failover is fast but doubles cost; cold (active-passive) is cheaper but slower to recover.
- Each additional 'nine' of availability requires disproportionately more engineering effort — don't over-provision without justification.

- SPOF elimination is layered: app servers, database, availability zone, and region each need their own redundancy plan.
- Tie the required availability target to concrete business impact, not to an arbitrary 'more nines is better' instinct.